4TH-YEAR MAJOR PROJECT & PRODUCT ARCHITECTURE BLUEPRINT (2026–27)

ACR-RAG: Adaptive Control & Retrieval Platform

A Dual-System (System 1 Jev + System 2 Agentic RAG) Self-Evaluating, Cost-Aware AI Product | Department of AI & Data Science, VIIT Pune

1. The Whole Project Idea in One Single Paragraph (With Example)

ACR-RAG (Adaptive Control & Retrieval Platform) is a plug-and-play, cost-aware AI product and research platform that replaces wasteful "one-size-fits-all" RAG pipelines with an ultra-fast **System 1 Calibrated Control Plane** (TypeSafe AI's Jev) that dynamically governs when and how an expensive **System 2 Retrieval & Multi-Agent Pipeline** (LangGraph, LightRAG, and Model Context Protocol (MCP)) is invoked—cutting token costs by 60–80%, eliminating unnecessary latency, and refusing to answer when verified evidence is missing. **For example**, when an external company onboards their workspace by uploading vendor SLA PDFs and connecting their live PostgreSQL database and GitHub repository via MCP, our system intercepts every incoming user query with **Jev** in ~90 milliseconds: if an engineer asks a simple question like *"What is our standard late-delivery penalty?"*, Jev's typed **`Choice`** primitive routes it to **Tier 1 (Fast Vector RAG)**, verifies chunk sufficiency using a calibrated **`Noul`** check (**`0.94`**), and answers in 0.7 seconds for $0.0002 using a small model; if they ask a comparative question like *"Compare penalty clauses between SteelCorp and Apex Logistics,"* Jev selects **Tier 2 (Hybrid BM25 + Vector Search + Cross-Encoder Reranking)**; and if they ask a complex operational question like *"Why is the Pune Metro Phase-3 assembly delayed, which suppliers are responsible, and which open GitHub blockers are tied to their shipments?"*, Jev either routes directly to **Tier 3 (Agentic RAG)** or automatically escalates mid-flight when initial retrieval returns a low **`Noul`** confidence score (**`0.39`**), launching a LangGraph supervisor that traverses the LightRAG knowledge graph, queries live SQL shipment tables and GitHub issues via FastMCP, verifies every generated sentence against source citations before responding, and logs the complete execution trace, RAGAS faithfulness score, and exact dollar savings into the **Agent Control Room (ACR)** dashboard.

---

2. Problem Statement: Why Existing RAG Pipelines & College Projects Fail

Modern enterprises store critical knowledge across unstructured PDFs, policy wikis, relational SQL databases, and live issue trackers. While Retrieval-Augmented Generation (RAG) and AI Agents are widely used, both production teams and student projects face four fundamental bottlenecks:

- **The "One-Pipeline-Fits-All" Cost & Latency Bottleneck:** Standard RAG systems execute the exact same pipeline for every prompt. Using basic Vector RAG on multi-hop questions fails because vector similarity cannot traverse cross-document entity relationships or query live SQL state. Conversely, running an "Always-On Agentic GraphRAG" pipeline on simple FAQ queries wastes 10x–30x more tokens and adds 8–20 seconds of latency.
- **The Router Paradox (Using a Slow LLM to Route an LLM):** Recent academic approaches like Adaptive-RAG and Corrective RAG (CRAG) propose routing queries by complexity, but most implementations call an autoregressive LLM to classify queries or critique chunks. Making three slow LLM calls to avoid one LLM call defeats the latency and cost objective.
- **Uncalibrated Confidence & Hallucinations on Missing Evidence:** Autoregressive LLMs trained with RLHF are notoriously overconfident when asked to estimate their own certainty. When a knowledge

base lacks sufficient evidence (e.g., *"Which supplier will win next quarter's tender?"*), standard RAG pipelines guess and hallucinate rather than abstaining cleanly.

- **The "College Project Trap" (Static Demo vs. Usable Product):** 95% of final-year student projects hardcode one static folder of PDFs into a local Python script. External developers or companies cannot upload their own knowledge, connect their own databases, tune cost/accuracy thresholds, or consume the system via a standard API key.

3. Our Core Innovation: Dual-System Architecture & "Project-to-Product" Transformation

Inspired by Daniel Kahneman's cognitive framework (*Thinking, Fast and Slow*), ACR-RAG separates **fast, calibrated software decisions (System 1)** from **deep, expressive text synthesis and multi-step tool reasoning (System 2)**, wrapped inside a self-serve developer product.

3.1 System 1 Control Plane (TypeSafe AI's Jev) vs. System 2 Generation Plane (LLMs + LangGraph)

Instead of prompting a text-generating LLM to output JSON routing decisions, our control plane uses TypeSafe AI's Jev—the first **System One model** trained via **Reinforcement Learning for Calibrated Decisions (RLCD)**. Jev does not generate chat strings token-by-token; it takes program state plus a batch of atomic typed questions (`Choice`, `Score`, and `Noul`) and evaluates all of them **in parallel in 70–200 ms** at **$0.042 per million input tokens (and free output tokens)**, returning schema-guaranteed answers with mathematically calibrated probabilities.

Control Checkpoint	Jev Primitive Used	What It Evaluates in Parallel (~90 ms)	Deterministic Action Taken by Code
1. Pre-Retrieval Intent & Complexity Router	`Choice` + `Score` + `Noul`	Selects tier (`low_vector`, `med_hybrid`, `high_agentic`), scores entity/clause complexity (1–5), and checks `Noul`: *"Requires live external DB/API?"*	Routes query to minimum viable retrieval & model tier (e.g., Tier 1 with small fast LLM vs. Tier 3 LangGraph Supervisor).
2. Mid-Pipeline Confidence Gate	`Score` + `Noul (0–1)`	Evaluates retrieved chunks against query: `Noul` (*"Do passages contain complete facts to answer?"*) + combines with Cross-Encoder logit margin.	If `C_evidence >= Threshold`, synthesize immediately. If `< Threshold`, trigger **Confidence-Gated Escalation** to Tier 2/3.
3. Post-Generation Verifier & Abstention Guard	Parallel `Noul (0–1)` per sentence	Evaluates every generated claim against its cited document chunk or MCP tool output: *"Is claim X directly entailed by source Y?"*	Strips or flags unsupported claims; triggers formal **Abstention** if post-escalation evidence remains below `0.50`.

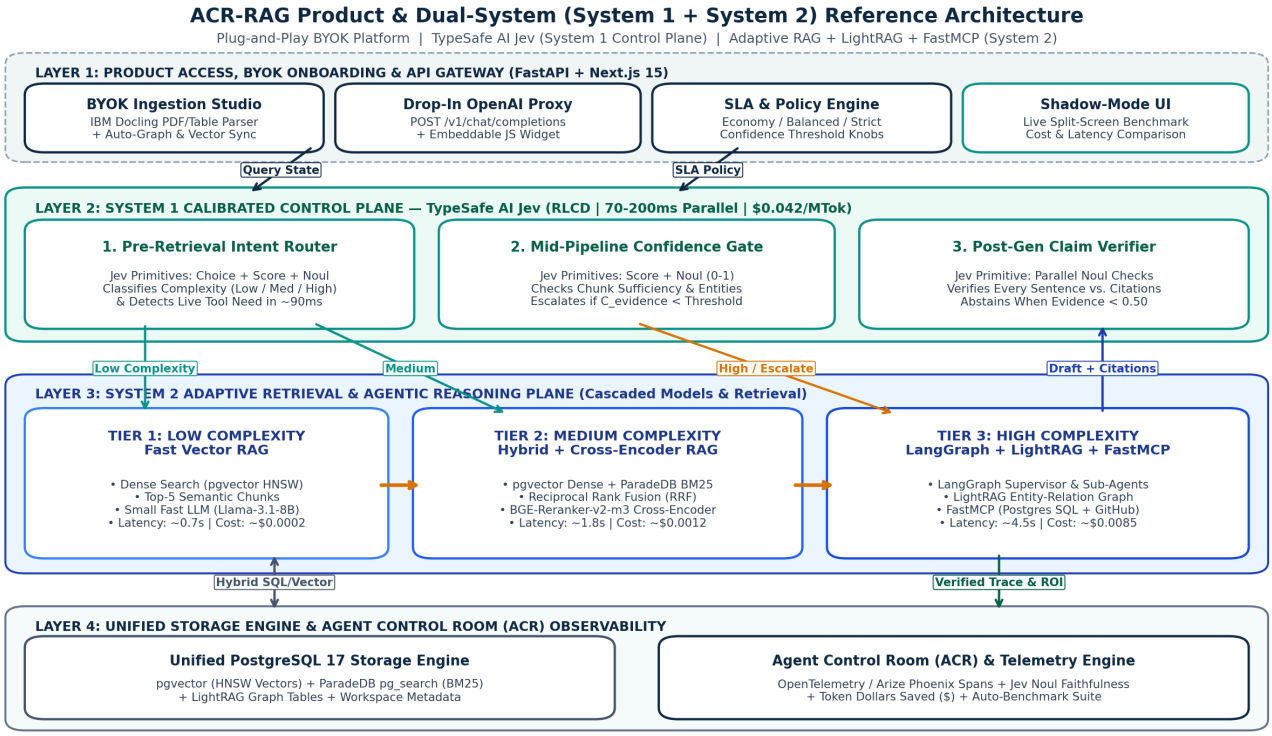


Figure 1: Complete 4-Layer Product & Dual-System Architecture of ACR-RAG combining BYOK Product Onboarding, TypeSafe AI Jev (System 1 Control Plane), 3-Tier Adaptive Retrieval/LangGraph (System 2), and Unified PostgreSQL 17 + ACR Observability.

3.2 Four Features That Turn ACR-RAG From a College Project Into a Real Product

1. **3-Click "Bring Your Own Knowledge" (BYOK) Workspace Studio:** Any external team or developer can create an isolated Workspace, drag-and-drop PDFs/DOCX/Markdown files (automatically parsed via IBM Docling into vector, BM25, and graph indexes), and toggle pre-built FastMCP connectors by pasting a Read-Only PostgreSQL URI or GitHub Repository token.
2. **OpenAI-Compatible Drop-In Proxy API (`POST /v1/chat/completions`) & Web Widget:** External software products do not need to learn a new SDK. They point their existing OpenAI client to **`base_url="https://acr-rag.yourdomain.com/v1"`** with their Workspace API Key. ACR-RAG performs adaptive routing behind the scenes and injects telemetry into HTTP response headers (**`X-ACR-Tier-Used`**, **`X-ACR-Jev-Confidence`**, **`X-ACR-Latency-Ms`**, **`X-ACR-Cost-Saved-USD`**).
3. **Interactive SLA & Cost Governance Policy Sliders:** Workspace owners control system behavior in real time via three operating profiles in the Agent Control Room: **Economy Mode** (escalates only when confidence **`< 0.35`**), **Balanced Mode** (escalates when **`< 0.65`**), and **Strict Zero-Hallucination Mode** (requires **`Noul >= 0.88`** on every sentence or abstains).
4. **One-Click "Auto-Benchmark My Workspace" & Shadow Mode Playground:** When a user uploads documents, clicking *"Run Workspace Benchmark"* automatically synthesizes 15 test queries across complexity tiers and runs a side-by-side comparison of **Always-On Agentic RAG vs. ACR-RAG**, proving exact latency and token cost savings on the user's own data.

---

4. Latest 2026 Researched Technology Stack (Practical, Fast & Defensible)

We audited the 2026 AI engineering ecosystem to eliminate brittle legacy dependencies (such as separate Elasticsearch clusters, heavy Neo4j servers, or naive PyPDF parsers) in favor of a unified, high-performance stack that a 4-person team can build and deploy reliably.

Layer	Selected 2026 Technology	What It Replaces (Legacy)	Why We Chose It for ACR-RAG
System 1 Router & Verifier	TypeSafe AI Jev (`jev-preview`) + RouteLLM fallback	Prompted GPT-4 / Claude JSON routers (slow, expensive, uncalibrated)	Evaluates parallel `Choice`, `Score`, and `Noul` primitives in 70–200 ms at $0.042/MTok with calibrated probabilities and 0% schema error.
Document Parsing & Chunking	IBM Docling (Open-Source Document Intelligence)	PyPDF2 / Unstructured basic text splitters	Preserves complex tables, multi-column layouts, section hierarchy, and page provenance—critical for accurate contract/SLA citations.
Unified Vector + BM25 Database	**PostgreSQL 17** + pgvector (HNSW) + ParadeDB pg_search	Separate Pinecone + Elasticsearch + Postgres instances	Runs dense vector search, Rust-based BM25 full-text search, Reciprocal Rank Fusion (RRF), and relational storage inside **one Postgres DB**.
Embeddings & Cross-Encoder	BAAI/bge-m3 + `bge-reranker-v2-m3`	OpenAI `text-embedding-ada-002` (paid, dense-only)	Open-weight multilingual dense + sparse retrieval with high-precision Cross-Encoder logit margins for our confidence formula.
GraphRAG Engine	LightRAG (EMNLP 2025/2026 Dual-Level GraphRAG)	Heavy Neo4j cluster + Microsoft GraphRAG (huge indexing token cost)	Extracts entity-relation graphs at 10x lower token cost than MS GraphRAG and stores graph topologies cleanly over Postgres/KV tables.
Agent Orchestration & Tools	LangGraph + FastMCP (MCP 2026 Spec)	Black-box LangChain `AgentExecutor` chains & custom REST wrappers	LangGraph gives cyclic state control for mid-flight escalation; FastMCP turns Python SQL & GitHub functions into standard MCP servers in 15 lines.
System 2 LLMs (Cascaded)	**Tier 1/2:** Llama-3.3-70B / GPT-4o-mini **Tier 3:** Claude Sonnet / GPT-5 class	Single expensive frontier model for every query	Combining retrieval routing with model cascading cuts token expenditure by 65–80% on mixed enterprise workloads.
Backend, UI & Observability	FastAPI + Next.js 15 (shadcn/ui) + Arize Phoenix + RAGAS	Streamlit toy UI with terminal `print()` debugging	Production SSE streaming, OpenTelemetry span tracing (`openinference`), and automated RAGAS evaluation scorecards in the Agent Control Room.

5. Complete User Flow & Runtime Technical Flow

5.1 Product User Flow: How External Teams & Developers Use ACR-RAG

- **Step 1 — Workspace Creation & 3-Click Knowledge Ingestion:** A user logs into the ACR-RAG Web Studio and creates a workspace (e.g., `Pune-Manufacturing-Ops`). They upload PDF contracts and engineering manuals. In the background, **IBM Docling** extracts markdown + tables, **BGE-M3** indexes chunks into `pgvector` + `pg_search`, and **LightRAG** builds the entity-relationship graph.

- **Step 2 — Attaching Live External Systems via FastMCP:** In the *Connectors* tab, the user toggles on the **PostgreSQL MCP Server** (providing a read-only connection string to their ERP/inventory table) and the **GitHub MCP Server** (providing a repo token for issue/PR tracking).
- **Step 3 — Setting the Governance Policy & Running Auto-Benchmark:** The user selects **Balanced SLA Mode** (escalation threshold `0.65`, abstention threshold `0.50`) and clicks *"Auto-Benchmark Workspace"* to view their baseline Pareto chart of Cost vs. Faithfulness.
- **Step 4 — Live Consumption (Playground, Web Widget, or Drop-In API):** Team members ask questions in the split-screen Shadow Mode Playground, or engineers call `POST /v1/chat/completions` from their internal ERP portal. Every query records a live trace in the **Agent Control Room**.

5.2 Step-by-Step Technical Execution Flow & Mathematical Confidence Formula

When a request hits `POST /v1/chat/completions`, the FastAPI orchestrator executes a deterministic 6-stage state machine:

1. **Stage 1 — Pre-Retrieval System-1 Triage (~90 ms):** FastAPI sends the user query and workspace tool manifest to **Jev** in a single parallel request evaluating three primitives:
   - `tier_choice = Choice(["tier1_vector", "tier2_hybrid", "tier3_agentic"])`
   - `needs_live_mcp = Noul("Does answering this question require live database records, issue status, or external tool state?")`
   - `reasoning_depth = Score(rubric="1: Single fact lookup; 3: Multi-document comparison; 5: Multi-hop causal or relational synthesis")`
2. **Stage 2 — Minimum Viable Retrieval Execution:** Based on Jev's output, the router invokes the lowest-cost tier capable of handling the query (Tier 1 HNSW vector search for simple lookups; Tier 2 Reciprocal Rank Fusion of `pgvector` + ParadeDB `pg_search` BM25 with `bge-reranker-v2-m3` Cross-Encoder reranking for comparisons).
3. **Stage 3 — Mathematical Evidence Sufficiency Gate (C_{evidence}):** Before generating an answer (or deciding whether to escalate from Tier 1/2 to Tier 3), the system computes a composite evidence confidence score:

   $$C_{\text{evidence}} = 0.50 \cdot P_{\text{Jev_Noul}}(\text{Sufficient}) + 0.30 \cdot \sigma(\Delta L_{\text{rerank}}) + 0.20 \cdot \text{Cov}_{\text{entities}}$$

   where $P_{\text{Jev_Noul}}(\text{Sufficient})$ is Jev's calibrated `Noul` probability that the top-K chunks completely answer the query, $\sigma(\Delta L_{\text{rerank}})$ is the normalized logit margin between the top-1 and top-5 Cross-Encoder scores, and $\text{Cov}_{\text{entities}}$ is the fraction of query named entities present in the retrieved context.
4. **Stage 4 — Confidence-Gated Escalation to Tier 3 (LangGraph + LightRAG + FastMCP):** If $C_{\text{evidence}} < \tau_{\text{escalate}}$ (e.g., `0.65` in Balanced Mode)—or if Stage 1 directly selected Tier 3—control transitions into the **LangGraph Supervisor**. The supervisor decomposes the query into sub-goals, traverses 2-hop entity links in **LightRAG**, and invokes **FastMCP tools** (SQL queries, GitHub issue lookups).
5. **Stage 5 — Calibrated Abstention ("Don't Answer If Evidence Is Insufficient"):** After Tier 3 completes retrieval, Jev re-evaluates C_{evidence}. If the score remains below τ_{abstain} (`0.50`), the pipeline halts generation and returns a structured **Insufficient Evidence Response** explicitly listing which facts or documents are missing from the workspace.
6. **Stage 6 — Post-Generation Claim Verification & ACR Telemetry:** When a draft answer with inline citations (`[Doc-1, p.4]`, `[MCP-SQL]`) is produced, Jev evaluates a parallel batch of `Noul` checks across each sentence. Verified responses stream to the client via SSE while an async task writes token usage, latency, routing path, and faithfulness scores to PostgreSQL and **Arize Phoenix**.

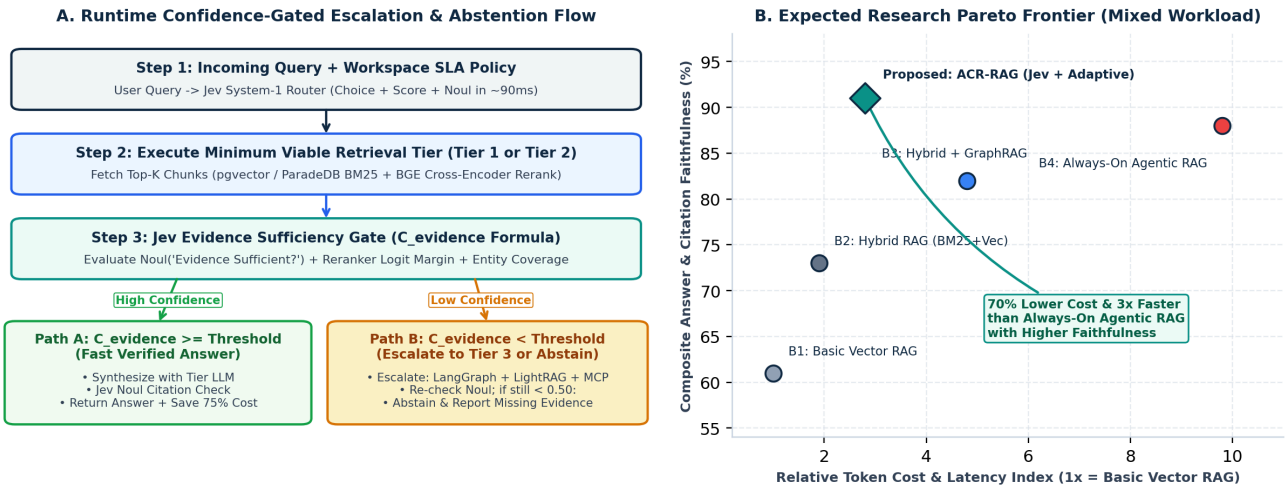


Figure 2: (Left) Runtime Confidence-Gated Escalation and Calibrated Abstention State Machine; (Right) Expected Research Evaluation Pareto Frontier comparing ACR-RAG against four fixed baselines on mixed-complexity workloads.

---

6. Concrete Real-Life End-to-End Example (Shareable Team Scenario)

To make the project immediately understandable for your teammates and exam panel, consider a real-world deployment for **Apex AutoComponents Pvt. Ltd. (Pune)**, a mid-sized EV battery-pack manufacturer. The company creates a workspace in ACR-RAG and connects three knowledge sources in 3 minutes:

- **Source 1 (Unstructured PDFs via IBM Docling):** 45 Supplier SLA Contracts, Quality Assurance SOPs, and HR/Operations Policies indexed into **`pgvector`** + ParadeDB **`pg_search`**.
- **Source 2 (Entity Graph via LightRAG):** Automatically extracted graph linking **`Projects → Battery Modules → Raw Material Suppliers → Penalty Clauses → Engineering Teams`**.
- **Source 3 (Live FastMCP Servers):** A Read-Only **PostgreSQL ERP MCP tool** (live shipment delays & batch defect rates) and a **GitHub MCP tool** (open firmware/hardware engineering issues).

During a single shift, the Operations Manager submits four different questions through the workspace chat portal. Instead of running the same expensive agent loop four times, ACR-RAG dynamically adapts its execution depth per query—answering the simple SOP lookup in 0.68 seconds, automatically escalating mid-flight when a contract query requires live ERP batch data, orchestrating multi-hop Graph + GitHub MCP reasoning for cross-team blockers, and cleanly abstaining when asked an unanswerable future tender question:

User Query	Stage 1: Jev Router Decision (~90 ms)	Stage 2–5: Execution Path & Confidence Gate	Final Outcome, Latency & Cost
Query 1 (Simple): *"What is the maximum defect tolerance percentage allowed for lithium cell batches?"*	**`Choice`**: **`tier1_vector`** (0.96) **`needs_mcp`**: **`0.02`** **`Score`**: **`1.1 / 5.0`**	Fetches top-3 chunks from **`QA_SOP_2026.pdf`** via **`pgvector`**. Jev **`Noul`** sufficiency check returns **0.95** (≥ 0.65). Synthesizes with Llama-3.1-8B.	**Answered & Verified** Citations: **`[QA_SOP, p.12]`** Latency: **0.68 sec** Cost: **$0.00018** (Saved 97%)

User Query	Stage 1: Jev Router Decision (~90 ms)	Stage 2–5: Execution Path & Confidence Gate	Final Outcome, Latency & Cost
Query 2 (Medium + Mid-Flight Escalation): *"Why was the EV-Pack-V2 line halted yesterday, and did the supplier breach their SLA penalty window?"*	**`Choice`**: **`tier2_hybrid`** (0.61) **`needs_mcp`**: **`0.48`** *(Initially tries Tier 2 Hybrid RAG)*	Tier 2 retrieves SLA PDF chunks, but finds no data on "yesterday's halt". Jev mid-pipeline **`Noul`** returns **0.34 (< 0.65) → Auto-Escalates to Tier 3!** LangGraph queries LightRAG (finds supplier *VoltCore India*), calls Postgres ERP MCP (finds 9-day delay on Batch #402), and matches SLA Clause 8.2 (7-day cap).	**Escalated & Verified** Citations: **`[MCP-ERP #402]`**, **`[VoltCore_SLA, p.6]`** Latency: **3.90 sec** Cost: **$0.0064**
Query 3 (Complex Multi-Hop): *"Which open GitHub firmware issues affect projects using VoltCore's delayed batches, and who is the lead engineer?"*	**`Choice`**: **`tier3_agentic`** (0.94) **`needs_mcp`**: **`0.97`** **`Score`**: **`4.8 / 5.0`**	Directly launches Tier 3 LangGraph Supervisor: (1) LightRAG links *VoltCore → BMS-Controller-v4 → Lead: R. Kulkarni*; (2) GitHub MCP fetches open issues **#114** and **#119**; (3) Jev verifies all 4 claims in parallel (**`Noul = 0.96`**).	**Answered & Verified** Citations: **`[Graph: BMS-v4]`**, **`[GitHub #114, #119]`** Latency: **4.45 sec** Cost: **$0.0079**
Query 4 (Unanswerable / Trap): *"Which supplier will win our Q3 2027 cobalt tender and at what price per ton?"*	**`Choice`**: **`tier2_hybrid`** (0.72) **`needs_mcp`**: **`0.31`**	Tier 2 retrieves past 2025/2026 tender docs. Jev **`Noul`** returns **0.11**. Escalates to Tier 3 SQL/Graph search; post-escalation **`Noul`** remains **0.14 (< 0.50 Abstention Floor)**.	**Calibrated Abstention** Refuses to guess; states Q3 2027 tender bids are not yet in ERP or docs. Hallucination: **0%**

7. Team Execution Plan (4-Member Split) & Research Paper Blueprint

So that all four team members can work in parallel on clean interfaces without blocking each other, divide the repository into four modular packages plus a shared 5-baseline experimental evaluation for your research paper:

Team Member Role	Core Modules Owned	Key Technologies & Files	Research Paper Contribution
Member 1: Control Plane & Router Lead	Pre-Retrieval Jev Router, Mid-Pipeline Confidence Gate (C_{evidence}), Post-Gen **`Noul`** Verifier & Abstention Engine.	TypeSafe AI Jev SDK, RouteLLM/DeBERTa baseline, calibration formulas (**`router/`**, **`verifier/`**).	Authors Section 3 (System 1 Routing & Confidence Calibration Math) + Calibration curves.
Member 2: Data, Hybrid RAG & GraphRAG Lead	BYOK Ingestion Pipeline, Unified Postgres Vector + BM25 RRF Search, Cross-Encoder Reranking, LightRAG Graph Indexer.	IBM Docling, PostgreSQL 17, **`pgvector`**, ParadeDB **`pg_search`**, **`BAAI/bge-m3`**, LightRAG (**`retrieval/`**).	Authors Section 4 (Hybrid + Graph Retrieval Architecture) + Retrieval Recall@K ablation tables.
Member 3: Agentic Workflow & FastMCP Lead	Tier 3 LangGraph Supervisor, Read-Only Postgres SQL MCP Server, GitHub Issues MCP Server, OpenAI Proxy API.	LangGraph, FastMCP, FastAPI SSE streaming, Pydantic v2 (**`agents/`**, **`mcp_servers/`**, **`api/`**).	Authors Section 5 (Agentic Escalation & MCP Tool Orchestration) + Multi-hop task completion analysis.

Team Member Role	Core Modules Owned	Key Technologies & Files	Research Paper Contribution
Member 4: Product UI, Control Room & Eval Lead	Next.js BYOK Studio, Split-Screen Shadow Mode Playground, Agent Control Room Dashboard, Auto-Benchmark Suite.	Next.js 15, Tailwind, shadcn/ui, Recharts, Arize Phoenix OpenTelemetry, RAGAS (`frontend/`, `evals/`).	Authors Section 6 (Experimental Setup, 5-Baseline Pareto Plots, Latency & Cost Analysis).

How to Validate This for Your IEEE / Springer / ACM Research Paper

Run a controlled 300-query benchmark (100 simple single-hop, 100 multi-document comparative, 70 multi-hop relational/tool queries, and 30 unanswerable trap queries) across **five configurations**: **(B1)** Basic Vector RAG, **(B2)** Hybrid BM25+Vector RAG, **(B3)** Hybrid + LightRAG, **(B4)** Always-On LangGraph Agentic RAG, and **(Proposed)** ACR-RAG with Jev System-1 Control. Report **Answer Faithfulness**, **Citation Precision**, **Abstention F1 on Unanswerable Queries**, **P50/P95 Latency (ms)**, and **Total Cost per 1,000 Queries ($)**. Demonstrating that ACR-RAG matches or beats Always-On Agentic RAG in faithfulness while cutting token costs by ~70% and latency by ~3x gives you an undeniable, publication-grade result.